Lab Report No 11
Understanding of Transfer Learning
Artificial Intelligence

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

2nd Jun,2025

Department of Computer Engineering,

HITEC University, Taxila

Example Task No 1:

Solution:

Brief description (3-5 lines)
Consider you want to build a model to classify cats and dogs, but you only have a small dataset. You can use a pre-trained model like VGG16, trained on ImageNet, replace its final layer to output the number of cats and dogs classes, and then fine-tune this model on your flower dataset. This approach takes advantage of the robust features learned by VGG16 from the large ImageNet dataset, improving the performance of your flower classification task.
The code

```
try:
  # This command only in Colab.
  %tensorflow_version 2.x
except Exception:
  pass
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, Flatten, Dropout,
MaxPooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import numpy as np
import matplotlib.pyplot as plt
# Get project files
!wget https://cdn.freecodecamp.org/project-data/cats-and-
dogs/cats_and_dogs.zip
!unzip cats_and_dogs.zip
PATH = 'cats_and_dogs'
train_dir = os.path.join(PATH, 'train')
validation_dir = os.path.join(PATH, 'validation')
test_dir = os.path.join(PATH, 'test')
# Get number of files in each directory. The train and validation directories
# each have the subdirecories "dogs" and "cats".
total_train = sum([len(files) for r, d, files in os.walk(train_dir)])
total_val = sum([len(files) for r, d, files in os.walk(validation_dir)])
total_test = len(os.listdir(test_dir))
# Variables for pre-processing and training.
batch_size = 128
epochs = 15
```

```
IMG_HEIGHT = 150
IMG_WIDTH = 150
# 3
train_image_generator      = ImageDataGenerator(rescale=1./255)
validation_image_generator = ImageDataGenerator(rescale=1./255)
test_image_generator       = ImageDataGenerator(rescale=1./255)
train_data_gen = train_image_generator.flow_from_directory(
                 train_dir,
                 target_size=(IMG_HEIGHT, IMG_WIDTH),
                 batch_size=batch_size,
                 class_mode='binary')
val_data_gen   = validation_image_generator.flow_from_directory(
                 validation_dir,
                 target_size=(IMG_HEIGHT, IMG_WIDTH),
                 batch_size=batch_size,
                 class_mode='binary')
test_data_gen  = test_image_generator.flow_from_directory(
                 PATH,
                 target_size=(IMG_HEIGHT, IMG_WIDTH),
                 batch_size=batch_size,
                 classes=['test'],
                 shuffle=False)
# 4
def plotImages(images_arr, probabilities = False):
    fig, axes = plt.subplots(len(images_arr), 1, figsize=(5,len(images_arr) * 3))
    if probabilities is False:
      for img, ax in zip( images_arr, axes):
          ax.imshow(img)
          ax.axis('off')
    else:
      for img, probability, ax in zip( images_arr, probabilities, axes):
          ax.imshow(img)
          ax.axis('off')
          if probability > 0.5:
              ax.set_title("%.2f" % (probability*100) + "% dog")
          else:
              ax.set_title("%.2f" % ((1-probability)*100) + "% cat")
    plt.show()
sample_training_images, _ = next(train_data_gen)
plotImages(sample_training_images[:5])
```

```
# 5
train_image_generator = ImageDataGenerator(
                        rescale=1./255,
                        horizontal_flip=True,
                        rotation_range=20,
                        zoom_range=0.15,
                        width_shift_range=0.2,
                        height_shift_range=0.2,
                        shear_range=0.15,
                        fill_mode="nearest")
# 6
train_data_gen =
train_image_generator.flow_from_directory(batch_size=batch_size,
                                          directory=train_dir,
                                          target_size=(IMG_HEIGHT, IMG_WIDTH),
                                          class_mode='binary')
augmented_images = [train_data_gen[0][0][0] for i in range(5)]
plotImages(augmented_images)
# 7
from keras.layers import Input
model = Sequential()
# Convolutions
model.add(Input(shape=(IMG_HEIGHT, IMG_WIDTH, 3)))
model.add(Conv2D(32, (3,3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, (3,3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(128, (3,3)))
model.add(MaxPooling2D(pool_size=(2, 2)))
# Dense layers
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Optimizer
model.compile(optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy'])
model.summary()
# 8
history = model.fit(
```

```
          train_data_gen, #steps_per_epoch=train_steps,
          validation_data=val_data_gen, #validation_steps=val_steps,
          epochs=epochs)
# 9
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(epochs)
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
#10
probabilities = model.predict(test_data_gen).flatten()
probabilities
# 11
answers =  [1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0,
          1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0,
          1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1,
          1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1,
          0, 0, 0, 0, 0, 0]
correct = 0
for probability, answer in zip(probabilities, answers):
  if round(probability) == answer:
    correct +=1
percentage_identified = (correct / len(answers)) * 100
passed_challenge = percentage_identified >= 63
print(f"Your model correctly identified {round(percentage_identified, 2)}% of
the images of cats and dogs.")
if passed_challenge:
  print("You passed the challenge!")
```

```
else:
  print("You haven't passed yet. Your model should identify at least 63% of the images. Keep trying. You will get it!")
```

The results (Screenshot)

```
--2024-01-23 08:00:44--  https://cdn.freecodecamp.org/project-data/cats-and-dogs/cats_and_dogs.zip
Resolving cdn.freecodecamp.org (cdn.freecodecamp.org)... 172.67.70.149, 104.26.2.33, 104.26.3.33, ...
Connecting to cdn.freecodecamp.org (cdn.freecodecamp.org)|172.67.70.149|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 70702765 (67M) [application/zip]
Saving to: 'cats_and_dogs.zip'

cats_and_dogs.zip   100%[===================>]  67.43M  89.3MB/s    in 0.8s

2024-01-23 08:00:46 (89.3 MB/s) - 'cats_and_dogs.zip' saved [70702765/70702765]

Archive:  cats_and_dogs.zip
   creating: cats_and_dogs/
  inflating: cats_and_dogs/.DS_Store
   creating: __MACOSX/
   creating: __MACOSX/cats_and_dogs/
```

```
 creating: cats_and_dogs/test/
inflating: cats_and_dogs/test/48.jpg
 creating: __MACOSX/cats_and_dogs/test/
inflating: __MACOSX/cats_and_dogs/test/._48.jpg
inflating: cats_and_dogs/test/49.jpg
inflating: __MACOSX/cats_and_dogs/test/._49.jpg
inflating: cats_and_dogs/test/.DS_Store
inflating: __MACOSX/cats_and_dogs/test/._.DS_Store
inflating: cats_and_dogs/test/8.jpg
inflating: __MACOSX/cats_and_dogs/test/._8.jpg
inflating: cats_and_dogs/test/9.jpg
inflating: __MACOSX/cats_and_dogs/test/._9.jpg
inflating: cats_and_dogs/test/14.jpg
inflating: __MACOSX/cats_and_dogs/test/._14.jpg
inflating: cats_and_dogs/test/28.jpg
inflating: __MACOSX/cats_and_dogs/test/._28.jpg
inflating: cats_and_dogs/test/29.jpg
inflating: __MACOSX/cats_and_dogs/test/._29.jpg
inflating: cats_and_dogs/test/15.jpg
inflating: __MACOSX/cats_and_dogs/test/._15.jpg
```

```
inflating: cats_and_dogs/test/17.jpg
inflating: __MACOSX/cats_and_dogs/test/._17.jpg
inflating: cats_and_dogs/test/16.jpg
inflating: __MACOSX/cats_and_dogs/test/._16.jpg
inflating: cats_and_dogs/test/12.jpg
inflating: __MACOSX/cats_and_dogs/test/._12.jpg
inflating: cats_and_dogs/test/13.jpg
inflating: __MACOSX/cats_and_dogs/test/._13.jpg
inflating: cats_and_dogs/test/39.jpg
inflating: __MACOSX/cats_and_dogs/test/._39.jpg
inflating: cats_and_dogs/test/11.jpg
inflating: __MACOSX/cats_and_dogs/test/._11.jpg
inflating: cats_and_dogs/test/10.jpg
inflating: __MACOSX/cats_and_dogs/test/._10.jpg
inflating: cats_and_dogs/test/38.jpg
inflating: __MACOSX/cats_and_dogs/test/._38.jpg
inflating: cats_and_dogs/test/21.jpg
inflating: __MACOSX/cats_and_dogs/test/._21.jpg
inflating: cats_and_dogs/test/35.jpg
inflating: __MACOSX/cats_and_dogs/test/._35.jpg
```

```
inflating: cats_and_dogs/test/34.jpg
inflating: __MACOSX/cats_and_dogs/test/._34.jpg
inflating: cats_and_dogs/test/20.jpg
inflating: __MACOSX/cats_and_dogs/test/._20.jpg
inflating: cats_and_dogs/test/36.jpg
inflating: __MACOSX/cats_and_dogs/test/._36.jpg
inflating: cats_and_dogs/test/22.jpg
inflating: __MACOSX/cats_and_dogs/test/._22.jpg
inflating: cats_and_dogs/test/23.jpg
inflating: __MACOSX/cats_and_dogs/test/._23.jpg
inflating: cats_and_dogs/test/37.jpg
inflating: __MACOSX/cats_and_dogs/test/._37.jpg
inflating: cats_and_dogs/test/33.jpg
inflating: __MACOSX/cats_and_dogs/test/._33.jpg
inflating: cats_and_dogs/test/27.jpg
inflating: __MACOSX/cats_and_dogs/test/._27.jpg
inflating: cats_and_dogs/test/26.jpg
inflating: __MACOSX/cats_and_dogs/test/._26.jpg
inflating: cats_and_dogs/test/32.jpg
inflating: __MACOSX/cats_and_dogs/test/._32.jpg
```

```
inflating: cats_and_dogs/test/18.jpg
inflating: __MACOSX/cats_and_dogs/test/._18.jpg
inflating: cats_and_dogs/test/24.jpg
inflating: __MACOSX/cats_and_dogs/test/._24.jpg
inflating: cats_and_dogs/test/30.jpg
inflating: __MACOSX/cats_and_dogs/test/._30.jpg
inflating: cats_and_dogs/test/31.jpg
inflating: __MACOSX/cats_and_dogs/test/._31.jpg
inflating: cats_and_dogs/test/25.jpg
inflating: __MACOSX/cats_and_dogs/test/._25.jpg
inflating: cats_and_dogs/test/19.jpg
inflating: __MACOSX/cats_and_dogs/test/._19.jpg
inflating: cats_and_dogs/test/42.jpg
inflating: __MACOSX/cats_and_dogs/test/._42.jpg
inflating: cats_and_dogs/test/4.jpg
inflating: __MACOSX/cats_and_dogs/test/._4.jpg
inflating: cats_and_dogs/test/5.jpg
inflating: __MACOSX/cats_and_dogs/test/._5.jpg
```

```
inflating: cats_and_dogs/test/43.jpg
inflating: __MACOSX/cats_and_dogs/test/._43.jpg
inflating: cats_and_dogs/test/7.jpg
inflating: __MACOSX/cats_and_dogs/test/._7.jpg
inflating: cats_and_dogs/test/41.jpg
inflating: __MACOSX/cats_and_dogs/test/._41.jpg
inflating: cats_and_dogs/test/40.jpg
inflating: __MACOSX/cats_and_dogs/test/._40.jpg
inflating: cats_and_dogs/test/6.jpg
inflating: __MACOSX/cats_and_dogs/test/._6.jpg
inflating: cats_and_dogs/test/2.jpg
inflating: __MACOSX/cats_and_dogs/test/._2.jpg
inflating: cats_and_dogs/test/50.jpg
inflating: __MACOSX/cats_and_dogs/test/._50.jpg
inflating: cats_and_dogs/test/44.jpg
inflating: __MACOSX/cats_and_dogs/test/._44.jpg
inflating: cats_and_dogs/test/45.jpg
inflating: __MACOSX/cats_and_dogs/test/._45.jpg
inflating: cats_and_dogs/test/3.jpg
```

```
inflating: __MACOSX/cats_and_dogs/test/._3.jpg
inflating: cats_and_dogs/test/47.jpg
inflating: __MACOSX/cats_and_dogs/test/._47.jpg
inflating: cats_and_dogs/test/1.jpg
inflating: __MACOSX/cats_and_dogs/test/._1.jpg
inflating: cats_and_dogs/test/46.jpg
inflating: __MACOSX/cats_and_dogs/test/._46.jpg
inflating: __MACOSX/cats_and_dogs/._test
 creating: cats_and_dogs/train/
 creating: cats_and_dogs/train/dogs/
inflating: cats_and_dogs/train/dogs/dog.775.jpg
 creating: __MACOSX/cats_and_dogs/train/
 creating: __MACOSX/cats_and_dogs/train/dogs/
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.775.jpg
inflating: cats_and_dogs/train/dogs/dog.761.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.761.jpg
inflating: cats_and_dogs/train/dogs/dog.991.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.991.jpg
inflating: cats_and_dogs/train/dogs/dog.749.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.749.jpg
```

```
inflating: cats_and_dogs/train/dogs/dog.985.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.985.jpg
inflating: cats_and_dogs/train/dogs/dog.952.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.952.jpg
inflating: cats_and_dogs/train/dogs/dog.946.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.946.jpg
inflating: cats_and_dogs/train/dogs/dog.211.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.211.jpg
inflating: cats_and_dogs/train/dogs/dog.577.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.577.jpg
inflating: cats_and_dogs/train/dogs/dog.563.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.563.jpg
inflating: cats_and_dogs/train/dogs/dog.205.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.205.jpg
inflating: cats_and_dogs/train/dogs/dog.239.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.239.jpg
inflating: cats_and_dogs/train/dogs/dog.588.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.588.jpg
inflating: cats_and_dogs/train/dogs/dog.365.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.365.jpg
inflating: cats_and_dogs/train/dogs/dog.403.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.403.jpg
inflating: cats_and_dogs/train/dogs/dog.417.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.417.jpg
inflating: cats_and_dogs/train/dogs/dog.371.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.371.jpg
inflating: cats_and_dogs/train/dogs/dog.359.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.359.jpg
inflating: cats_and_dogs/train/dogs/dog.601.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.601.jpg
inflating: cats_and_dogs/train/dogs/dog.167.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.167.jpg
inflating: cats_and_dogs/train/dogs/dog.173.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.173.jpg
inflating: cats_and_dogs/train/dogs/dog.615.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.615.jpg
inflating: cats_and_dogs/train/dogs/dog.36.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.36.jpg
inflating: cats_and_dogs/train/dogs/dog.22.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.22.jpg
inflating: cats_and_dogs/train/dogs/dog.629.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.629.jpg
inflating: cats_and_dogs/train/dogs/dog.826.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.826.jpg
inflating: cats_and_dogs/train/dogs/dog.198.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.198.jpg
inflating: cats_and_dogs/train/dogs/dog.832.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.832.jpg
inflating: cats_and_dogs/train/dogs/dog.833.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.833.jpg
inflating: cats_and_dogs/train/dogs/dog.827.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.827.jpg
inflating: cats_and_dogs/train/dogs/dog.199.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.199.jpg
inflating: cats_and_dogs/train/dogs/dog.628.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.628.jpg
inflating: cats_and_dogs/train/dogs/dog.23.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.23.jpg
inflating: cats_and_dogs/train/dogs/dog.37.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.37.jpg
inflating: cats_and_dogs/train/dogs/dog.172.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.172.jpg
inflating: cats_and_dogs/train/dogs/dog.614.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.614.jpg
inflating: cats_and_dogs/train/dogs/dog.600.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.600.jpg
inflating: cats_and_dogs/train/dogs/dog.166.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.166.jpg
inflating: cats_and_dogs/train/dogs/dog.358.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.358.jpg
inflating: cats_and_dogs/train/dogs/dog.416.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.416.jpg
inflating: cats_and_dogs/train/dogs/dog.370.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.370.jpg
inflating: cats_and_dogs/train/dogs/dog.364.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.364.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.364.jpg
inflating: cats_and_dogs/train/dogs/dog.402.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.402.jpg
inflating: cats_and_dogs/train/dogs/dog.589.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.589.jpg
inflating: cats_and_dogs/train/dogs/dog.238.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.238.jpg
inflating: cats_and_dogs/train/dogs/dog.562.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.562.jpg
inflating: cats_and_dogs/train/dogs/dog.204.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.204.jpg
inflating: cats_and_dogs/train/dogs/dog.210.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.210.jpg
inflating: cats_and_dogs/train/dogs/dog.576.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.576.jpg
inflating: cats_and_dogs/train/dogs/dog.947.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.947.jpg
inflating: cats_and_dogs/train/dogs/dog.953.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.953.jpg
inflating: cats_and_dogs/train/dogs/dog.984.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.984.jpg
inflating: cats_and_dogs/train/dogs/dog.748.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.748.jpg
inflating: cats_and_dogs/train/dogs/dog.990.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.990.jpg
inflating: cats_and_dogs/train/dogs/dog.760.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.760.jpg
inflating: cats_and_dogs/train/dogs/dog.774.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.774.jpg
inflating: cats_and_dogs/train/dogs/dog.762.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.762.jpg
inflating: cats_and_dogs/train/dogs/dog.776.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.776.jpg
inflating: cats_and_dogs/train/dogs/dog.986.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.986.jpg
inflating: cats_and_dogs/train/dogs/dog.992.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.992.jpg
inflating: cats_and_dogs/train/dogs/dog.979.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.979.jpg
inflating: cats_and_dogs/train/dogs/dog.945.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.945.jpg
inflating: cats_and_dogs/train/dogs/dog.789.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.789.jpg
inflating: cats_and_dogs/train/dogs/dog.951.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.951.jpg
inflating: cats_and_dogs/train/dogs/dog.206.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.206.jpg
inflating: cats_and_dogs/train/dogs/dog.560.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.560.jpg
inflating: cats_and_dogs/train/dogs/dog.574.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.574.jpg
inflating: cats_and_dogs/train/dogs/dog.212.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.212.jpg
inflating: cats_and_dogs/train/dogs/dog.548.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.548.jpg
inflating: cats_and_dogs/train/dogs/dog.372.jpg
```

```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.372.jpg
inflating: cats_and_dogs/train/dogs/dog.414.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.414.jpg
inflating: cats_and_dogs/train/dogs/dog.400.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.400.jpg
inflating: cats_and_dogs/train/dogs/dog.366.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.366.jpg
inflating: cats_and_dogs/train/dogs/dog.428.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.428.jpg
inflating: cats_and_dogs/train/dogs/dog.399.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.399.jpg
inflating: cats_and_dogs/train/dogs/dog.616.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.616.jpg
inflating: cats_and_dogs/train/dogs/dog.170.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.170.jpg
inflating: cats_and_dogs/train/dogs/dog.164.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.164.jpg
inflating: cats_and_dogs/train/dogs/dog.602.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.602.jpg
```

```
inflating: cats_and_dogs/train/dogs/dog.21.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.21.jpg
inflating: cats_and_dogs/train/dogs/dog.158.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.158.jpg
inflating: cats_and_dogs/train/dogs/dog.35.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.35.jpg
inflating: cats_and_dogs/train/dogs/dog.819.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.819.jpg
inflating: cats_and_dogs/train/dogs/dog.831.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.831.jpg
inflating: cats_and_dogs/train/dogs/dog.825.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.825.jpg
inflating: cats_and_dogs/train/dogs/dog.824.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.824.jpg
inflating: cats_and_dogs/train/dogs/dog.830.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.830.jpg
inflating: cats_and_dogs/train/dogs/dog.818.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.818.jpg
inflating: cats_and_dogs/train/dogs/dog.159.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.159.jpg
```

```
inflating: cats_and_dogs/train/dogs/dog.34.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.34.jpg
inflating: cats_and_dogs/train/dogs/dog.20.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.20.jpg
inflating: cats_and_dogs/train/dogs/dog.165.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.165.jpg
inflating: cats_and_dogs/train/dogs/dog.603.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.603.jpg
inflating: cats_and_dogs/train/dogs/dog.617.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.617.jpg
inflating: cats_and_dogs/train/dogs/dog.171.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.171.jpg
inflating: cats_and_dogs/train/dogs/dog.398.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.398.jpg
inflating: cats_and_dogs/train/dogs/dog.429.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.429.jpg
inflating: cats_and_dogs/train/dogs/dog.401.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.401.jpg
inflating: cats_and_dogs/train/dogs/dog.367.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.367.jpg
```

```
inflating: cats_and_dogs/train/dogs/dog.373.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.373.jpg
inflating: cats_and_dogs/train/dogs/dog.415.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.415.jpg
inflating: cats_and_dogs/train/dogs/dog.549.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.549.jpg
inflating: cats_and_dogs/train/dogs/dog.575.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.575.jpg
inflating: cats_and_dogs/train/dogs/dog.213.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.213.jpg
inflating: cats_and_dogs/train/dogs/dog.207.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.207.jpg
inflating: cats_and_dogs/train/dogs/dog.561.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.561.jpg
inflating: cats_and_dogs/train/dogs/dog.950.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.950.jpg
inflating: cats_and_dogs/train/dogs/dog.788.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.788.jpg
```

```
inflating: cats_and_dogs/train/dogs/dog.944.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.944.jpg
inflating: cats_and_dogs/train/dogs/dog.978.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.978.jpg
inflating: cats_and_dogs/train/dogs/dog.993.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.993.jpg
inflating: cats_and_dogs/train/dogs/dog.987.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.987.jpg
inflating: cats_and_dogs/train/dogs/dog.777.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.777.jpg
inflating: cats_and_dogs/train/dogs/dog.763.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.763.jpg
inflating: cats_and_dogs/train/dogs/dog.983.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.983.jpg
inflating: cats_and_dogs/train/dogs/dog.997.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.997.jpg
inflating: cats_and_dogs/train/dogs/dog.767.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.767.jpg
inflating: cats_and_dogs/train/dogs/dog.773.jpg
```

```
inflating: cats_and_dogs/train/dogs/dog.798.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.798.jpg
inflating: cats_and_dogs/train/dogs/dog.940.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.940.jpg
inflating: cats_and_dogs/train/dogs/dog.954.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.954.jpg
inflating: cats_and_dogs/train/dogs/dog.968.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.968.jpg
inflating: cats_and_dogs/train/dogs/dog.559.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.559.jpg
inflating: cats_and_dogs/train/dogs/dog.565.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.565.jpg
inflating: cats_and_dogs/train/dogs/dog.203.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.203.jpg
inflating: cats_and_dogs/train/dogs/dog.217.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.217.jpg
inflating: cats_and_dogs/train/dogs/dog.571.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.571.jpg
inflating: cats_and_dogs/train/dogs/dog.439.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.439.jpg
inflating: cats_and_dogs/train/dogs/dog.411.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.411.jpg
inflating: cats_and_dogs/train/dogs/dog.377.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.377.jpg
inflating: cats_and_dogs/train/dogs/dog.363.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.363.jpg
inflating: cats_and_dogs/train/dogs/dog.405.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.405.jpg
inflating: cats_and_dogs/train/dogs/dog.388.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.388.jpg
inflating: cats_and_dogs/train/dogs/dog.149.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.149.jpg
inflating: cats_and_dogs/train/dogs/dog.24.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.24.jpg
inflating: cats_and_dogs/train/dogs/dog.30.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.30.jpg
```

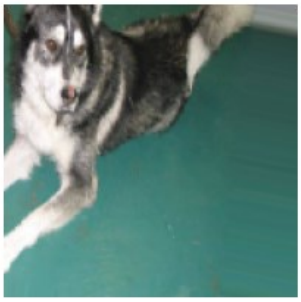
```
inflating: cats_and_dogs/train/dogs/dog.175.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.175.jpg
inflating: cats_and_dogs/train/dogs/dog.613.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.613.jpg
inflating: cats_and_dogs/train/dogs/dog.18.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.18.jpg
inflating: cats_and_dogs/train/dogs/dog.607.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.607.jpg
inflating: cats_and_dogs/train/dogs/dog.161.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.161.jpg
inflating: cats_and_dogs/train/dogs/dog.834.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.834.jpg
inflating: cats_and_dogs/train/dogs/dog.820.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.820.jpg
inflating: cats_and_dogs/train/dogs/dog.808.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.808.jpg
inflating: cats_and_dogs/train/dogs/dog.809.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.809.jpg
inflating: cats_and_dogs/train/dogs/dog.821.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.821.jpg
```

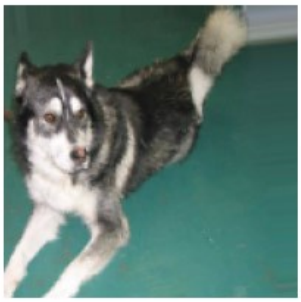
```
inflating: cats_and_dogs/train/dogs/dog.835.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.835.jpg
inflating: cats_and_dogs/train/dogs/dog.606.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.606.jpg
inflating: cats_and_dogs/train/dogs/dog.160.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.160.jpg
inflating: cats_and_dogs/train/dogs/dog.174.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.174.jpg
inflating: cats_and_dogs/train/dogs/dog.19.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.19.jpg
inflating: cats_and_dogs/train/dogs/dog.612.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.612.jpg
inflating: cats_and_dogs/train/dogs/dog.31.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.31.jpg
inflating: cats_and_dogs/train/dogs/dog.148.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.148.jpg
inflating: cats_and_dogs/train/dogs/dog.25.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.25.jpg
inflating: cats_and_dogs/train/dogs/dog.389.jpg
```

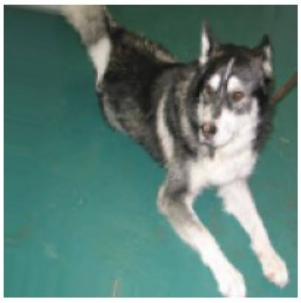
```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.389.jpg
inflating: cats_and_dogs/train/dogs/dog.362.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.362.jpg
inflating: cats_and_dogs/train/dogs/dog.404.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.404.jpg
inflating: cats_and_dogs/train/dogs/dog.410.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.410.jpg
inflating: cats_and_dogs/train/dogs/dog.376.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.376.jpg
inflating: cats_and_dogs/train/dogs/dog.438.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.438.jpg
inflating: cats_and_dogs/train/dogs/dog.216.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.216.jpg
inflating: cats_and_dogs/train/dogs/dog.570.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.570.jpg
inflating: cats_and_dogs/train/dogs/dog.564.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.564.jpg
inflating: cats_and_dogs/train/dogs/dog.202.jpg
```

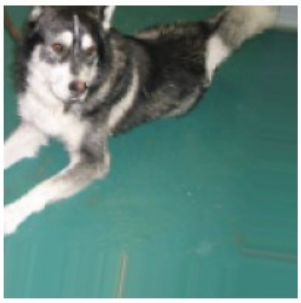
```
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.202.jpg
inflating: cats_and_dogs/train/dogs/dog.558.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.558.jpg
inflating: cats_and_dogs/train/dogs/dog.969.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.969.jpg
inflating: cats_and_dogs/train/dogs/dog.955.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.955.jpg
inflating: cats_and_dogs/train/dogs/dog.941.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.941.jpg
inflating: cats_and_dogs/train/dogs/dog.799.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.799.jpg
inflating: cats_and_dogs/train/dogs/dog.772.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.772.jpg
inflating: cats_and_dogs/train/dogs/dog.766.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.766.jpg
inflating: cats_and_dogs/train/dogs/dog.996.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.996.jpg
inflating: cats_and_dogs/train/dogs/dog.982.jpg
inflating: __MACOSX/cats_and_dogs/train/dogs/._dog.982.jpg
```

```
Found 2000 images belonging to 2 classes.
Found 1000 images belonging to 2 classes.
Found 50 images belonging to 1 classes.
```









```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 148, 148, 32)      896

 max_pooling2d (MaxPooling2  (None, 74, 74, 32)        0
 D)

 conv2d_1 (Conv2D)           (None, 72, 72, 64)        18496

 max_pooling2d_1 (MaxPoolin  (None, 36, 36, 64)        0
 g2D)

 conv2d_2 (Conv2D)           (None, 34, 34, 128)       73856

 max_pooling2d_2 (MaxPoolin  (None, 17, 17, 128)       0
 g2D)

 flatten (Flatten)           (None, 36992)             0

 dense (Dense)               (None, 512)               18940416

 dense_1 (Dense)             (None, 1)                 513

=================================================================
Total params: 19034177 (72.61 MB)
Trainable params: 19034177 (72.61 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
Epoch 1/15
16/16 [==============================] - 66s 4s/step - loss: 2.4797 - accuracy: 0.4915 -
val_loss: 0.7007 - val_accuracy: 0.5130
Epoch 2/15
16/16 [==============================] - 63s 4s/step - loss: 0.6932 - accuracy: 0.5280 -
val_loss: 0.6798 - val_accuracy: 0.5370
Epoch 3/15
16/16 [==============================] - 63s 4s/step - loss: 0.6837 - accuracy: 0.5535 -
val_loss: 0.6658 - val_accuracy: 0.6060
Epoch 4/15
16/16 [==============================] - 63s 4s/step - loss: 0.6819 - accuracy: 0.5665 -
val_loss: 0.6577 - val_accuracy: 0.6160
Epoch 5/15
16/16 [==============================] - 62s 4s/step - loss: 0.6744 - accuracy: 0.5785 -
val_loss: 0.6643 - val_accuracy: 0.5660
Epoch 6/15
16/16 [==============================] - 62s 4s/step - loss: 0.6821 - accuracy: 0.5595 -
val_loss: 0.6753 - val_accuracy: 0.5720
Epoch 7/15
16/16 [==============================] - 63s 4s/step - loss: 0.6792 - accuracy: 0.5655 -
val_loss: 0.6533 - val_accuracy: 0.5960
Epoch 8/15
16/16 [==============================] - 63s 4s/step - loss: 0.6697 - accuracy: 0.5715 -
val_loss: 0.6535 - val_accuracy: 0.6190
Epoch 9/15
16/16 [==============================] - 63s 4s/step - loss: 0.6623 - accuracy: 0.5950 -
val_loss: 0.6394 - val_accuracy: 0.6370
Epoch 10/15
16/16 [==============================] - 63s 4s/step - loss: 0.6573 - accuracy: 0.6120 -
val_loss: 0.6380 - val_accuracy: 0.6130
Epoch 11/15
16/16 [==============================] - 63s 4s/step - loss: 0.6547 - accuracy: 0.6040 -
val_loss: 0.6360 - val_accuracy: 0.6470
Epoch 12/15
16/16 [==============================] - 63s 4s/step - loss: 0.6524 - accuracy: 0.6060 -
val_loss: 0.6245 - val_accuracy: 0.6430
Epoch 13/15
16/16 [==============================] - 63s 4s/step - loss: 0.6528 - accuracy: 0.5940 -
val_loss: 0.7220 - val_accuracy: 0.5220
Epoch 14/15
16/16 [==============================] - 63s 4s/step - loss: 0.6522 - accuracy: 0.6030 -
val_loss: 0.6202 - val_accuracy: 0.6630
Epoch 15/15
16/16 [==============================] - 63s 4s/step - loss: 0.6461 - accuracy: 0.6260 -
val_loss: 0.6256 - val_accuracy: 0.6580
```


```
1/1 [==============================] - 1s 585ms/step

Out[10]:
array([0.34517658, 0.21194446, 0.42572957, 0.5968049 , 0.4792169 ,
       0.39588034, 0.53049237, 0.25247207, 0.43384677, 0.55174667,
       0.45386678, 0.4676831 , 0.28476182, 0.14748177, 0.34368533,
       0.41512853, 0.43693048, 0.49931747, 0.49519378, 0.44327635,
       0.34778595, 0.46220893, 0.25644436, 0.51871777, 0.42869654,
       0.4955555 , 0.6807136 , 0.22254878, 0.32284892, 0.3153577 ,
       0.39730492, 0.39650276, 0.5490188 , 0.90644455, 0.2894031 ,
       0.4413409 , 0.80926275, 0.5479006 , 0.33516577, 0.40053782,
       0.3734736 , 0.42286587, 0.65170896, 0.39839348, 0.55711496,
       0.37761185, 0.344579  , 0.38098595, 0.22960542, 0.38802674],
      dtype=float32)
```

```
Your model correctly identified 66.0% of the images of cats and dogs.
You passed the challenge!
```

Lab Task No 1:

Solution:

Brief description (3-5 lines)
Consider you want to build a model to classify types of flowers, but you only have a small dataset. You can use a pre-trained model like VGG16, trained on ImageNet, replace its final layer to output the number of flower classes, and then fine-tune this model on your flower dataset. This approach takes advantage of the robust features learned by VGG16 from the large ImageNet dataset, improving the performance of your flower classification task.
The code
 # Import Libraries import numpy as np import pandas as pd import os import warnings warnings.filterwarnings('ignore') import matplotlib.pyplot as plt from matplotlib.cm import get_cmap import seaborn as sns import random from collections import Counter import cv2 from tensorflow.keras.regularizers import l1, l2 from PIL import Image from sklearn.preprocessing import LabelEncoder from tensorflow.keras.models import Sequential from tensorflow.keras.preprocessing import image from tensorflow.keras.preprocessing.image import ImageDataGenerator from tensorflow.keras import layers, models from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout from tensorflow.keras.callbacks import EarlyStopping Load the dataset

```
data_train = '/kaggle/input/flowers-dataset/train'
flowers = {}
for dirname, dirlist, filenames in os.walk(data_train):
    flower_name = dirname.split('/')[-1]
    if dirname != data_train:
        print(f"Number of {flower_name} images: {len(filenames)}")
        filePaths = []
        for filename in filenames:
            filePaths.append(os.path.join(data_train, dirname, filename))
        flowers[flower_name] = filePaths
Display Sample of Images
fig, ax = plt.subplots(3, 3, figsize=(15, 15))
# Display images in the 3x3 grid
for i in range(3):
    for j in range(3):
        img = cv2.imread(flowers[list(flowers.keys())[0]][i*3 + j])
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        ax[i, j].imshow(img)
        # Hide axis for a cleaner look
        ax[i, j].axis('off')
plt.tight_layout()
plt.show()
class_names = os.listdir(data_train)
fig, axs = plt.subplots(len(class_names), 3, figsize=(15, len(class_names) * 5),
constrained_layout=True)
for i, cls in enumerate(class_names):
    cls_dir = os.path.join(data_train, cls)
    image_files = os.listdir(cls_dir)[:3]
    for j, img_name in enumerate(image_files):
        img_path = os.path.join(cls_dir, img_name)
        img = Image.open(img_path)
        axs[i, j].imshow(img)
        axs[i, j].axis('off')
        axs[i, j].set_title(cls)
plt.show()
Greyscale Images
fig, ax = plt.subplots(1,3, figsize=(24, 12))
img1 = cv2.imread(flowers[list(flowers.keys())[0]][0])
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
ax[0].imshow(img1)
```

```
img2 = cv2.imread(flowers[list(flowers.keys())[0]][1])
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
ax[1].imshow(img2)
img3 = cv2.imread(flowers[list(flowers.keys())[0]][3])
img3 = cv2.cvtColor(img3, cv2.COLOR_BGR2RGB)
ax[2].imshow(img3)
plt.show()
fig, ax = plt.subplots(1,3, figsize=(24, 12))
for idx, img in enumerate([img1, img2, img3]):
    img_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    ax[idx].imshow(img_gray, cmap='gray')
plt.show()
Count each image frequency
image_counts = {}
for cls in class_names:
    cls_dir = os.path.join(data_train, cls)
    num_images = len(os.listdir(cls_dir))
    image_counts[cls] = num_images
plt.figure(figsize=(12, 8))
bars = plt.bar(image_counts.keys(), image_counts.values(), color=['#a78889',
'#a8413d'])
plt.xlabel('Class', fontsize=14)
plt.ylabel('Number of Images', fontsize=14)
plt.title('Number of Images in Each Class', fontsize=16)
plt.xticks(rotation=45, ha='right', fontsize=12)
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height, f'{height}', ha='center',
va='bottom', fontsize=12)
plt.tight_layout()
plt.show()
Image Processing
#Edge Detection
fig, ax = plt.subplots(3, 3, figsize=(24, 24))
# Flatten the 2D axis array
axes = ax.flatten()
# Retrieve 9 unique images from the flowers dictionary
image_paths = []
for flower_name, paths in flowers.items():
    image_paths.extend(paths[:3])
```

```
# Extract 9 images from the paths
images_to_display = image_paths[:9]
# Apply Canny edge detection and display
for idx, img_path in enumerate(images_to_display):
    img = cv2.imread(img_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # Convert to RGB
    # Apply Canny edge detection
    img_edges = cv2.Canny(img_rgb, threshold1=127, threshold2=127)
    axes[idx].imshow(img_edges, cmap='gray')
    axes[idx].axis('off')
plt.tight_layout()
plt.show()
def apply_gradient(image_path):
    image = Image.open(image_path)
    image_array = np.array(image)
    height = image_array.shape[0]
    # Create a vertical gradient effect
    gradient = np.linspace(0, 1, height)
    gradient = gradient[:, np.newaxis]
    if image_array.ndim == 3:
        # Color image (RGB)
        gradient = gradient.reshape(height, 1, 1)
        gradient_image = image_array * gradient
        # Grayscale image
    else:
        gradient_image = image_array * gradient
    # Clip values to be in the valid range (0-255)
    gradient_image = np.clip(gradient_image, 0, 255).astype(np.uint8)
    # Convert back to an image
    return Image.fromarray(gradient_image)
plt.figure(figsize=(15, 10))
for i, img_path in enumerate(images_to_display):
    gradient_img = apply_gradient(img_path)
    plt.subplot(1, len(images_to_display), i + 1)
    plt.imshow(gradient_img)
    plt.axis('off')
    plt.title(f'Gradient Image {i + 1}')
plt.tight_layout()
plt.show()
# Blend Images
```

```
image1 = cv2.imread(images_to_display[0])
image2 = cv2.imread(images_to_display[1])
# Resize the second image to match the first image size
if image1.shape != image2.shape:
    image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))
# Blend the images
blended_image = cv2.addWeighted(image1, 0.5, image2, 0.5, 0)
plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(blended_image, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.title('Blended Image')
plt.show()
# Adjusting Contrast
# Contrast control
alpha = 1.5
# Brightness control
beta = 20
adjusted_image1 = cv2.convertScaleAbs(image1, alpha=alpha, beta=beta)
adjusted_image2 = cv2.convertScaleAbs(image2, alpha=alpha, beta=beta)
plt.figure(figsize=(12, 6))
# Display original and adjusted images
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))  # Convert BGR to
RGB for display
plt.title('Original Image 1')
plt.axis('off')
plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(adjusted_image1, cv2.COLOR_BGR2RGB))
plt.title('Adjusted Image 1')
plt.axis('off')
plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title('Original Image 2')
plt.axis('off')
plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(adjusted_image2, cv2.COLOR_BGR2RGB))
plt.title('Adjusted Image 2')
plt.axis('off')
plt.tight_layout()
plt.show()
```

```
# Zooming & Shrinking
zoom_factor = 1.7
shrink_factor = 0.3
# Zoom in (larger size)
zoomed_image1 = cv2.resize(image1, None, fx=zoom_factor, fy=zoom_factor,
interpolation=cv2.INTER_LINEAR)
zoomed_image2 = cv2.resize(image2, None, fx=zoom_factor, fy=zoom_factor,
interpolation=cv2.INTER_LINEAR)
# Shrink (smaller size)
shrunk_image1 = cv2.resize(image1, None, fx=shrink_factor, fy=shrink_factor,
interpolation=cv2.INTER_LINEAR)
shrunk_image2 = cv2.resize(image2, None, fx=shrink_factor, fy=shrink_factor,
interpolation=cv2.INTER_LINEAR)
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title('Original Image 1')
plt.axis('off')
plt.subplot(2, 3, 2)
plt.imshow(cv2.cvtColor(zoomed_image1, cv2.COLOR_BGR2RGB))
plt.title('Zoomed Image 1')
plt.axis('off')
plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(shrunk_image1, cv2.COLOR_BGR2RGB))
plt.title('Shrunk Image 1')
plt.axis('off')
plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title('Original Image 2')
plt.axis('off')
plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(zoomed_image2, cv2.COLOR_BGR2RGB))
plt.title('Zoomed Image 2')
plt.axis('off')
plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(shrunk_image2, cv2.COLOR_BGR2RGB))
plt.title('Shrunk Image 2')
plt.axis('off')
plt.tight_layout()
plt.show()
```

```
# Apply Gaussian Blur to reduce noise
gaussian_blurred_image1 = cv2.GaussianBlur(image1, (5, 5), 0)
gaussian_blurred_image2 = cv2.GaussianBlur(image2, (5, 5), 0)
# Apply Median Blur to reduce noise
median_blurred_image1 = cv2.medianBlur(image1, 5)
median_blurred_image2 = cv2.medianBlur(image2, 5)
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title('Original Image 1')
plt.axis('off')
plt.subplot(2, 3, 2)
plt.imshow(cv2.cvtColor(gaussian_blurred_image1, cv2.COLOR_BGR2RGB))
plt.title('Gaussian Blurred Image 1')
plt.axis('off')
plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(median_blurred_image1, cv2.COLOR_BGR2RGB))
plt.title('Median Blurred Image 1')
plt.axis('off')
plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title('Original Image 2')
plt.axis('off')
plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(gaussian_blurred_image2, cv2.COLOR_BGR2RGB))
plt.title('Gaussian Blurred Image 2')
plt.axis('off')
plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(median_blurred_image2, cv2.COLOR_BGR2RGB))
plt.title('Median Blurred Image 2')
plt.axis('off')
plt.tight_layout()
plt.show()
#Brightness Histogram & Histogram Equalization
gray_image1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray_image2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)
equalized_image1 = cv2.equalizeHist(gray_image1)
equalized_image2 = cv2.equalizeHist(gray_image2)
plt.figure(figsize=(15, 10))
# image 1
```

```
plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
plt.title('Original Image 1')
plt.axis('off')
plt.subplot(2, 3, 2)
plt.hist(gray_image1.ravel(), bins=256, range=[0, 256], color='gray')
plt.title('Brightness Histogram 1')
plt.xlim([0, 256])
plt.subplot(2, 3, 3)
plt.imshow(equalized_image1, cmap='gray')
plt.title('Equalized Image 1')
plt.axis('off')
# image 2
plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
plt.title('Original Image 2')
plt.axis('off')
plt.subplot(2, 3, 5)
plt.hist(gray_image2.ravel(), bins=256, range=[0, 256], color='gray')
plt.title('Brightness Histogram 2')
plt.xlim([0, 256])
plt.subplot(2, 3, 6)
plt.imshow(equalized_image2, cmap='gray')
plt.title('Equalized Image 2')
plt.axis('off')
plt.tight_layout()
plt.show()
```

CNN

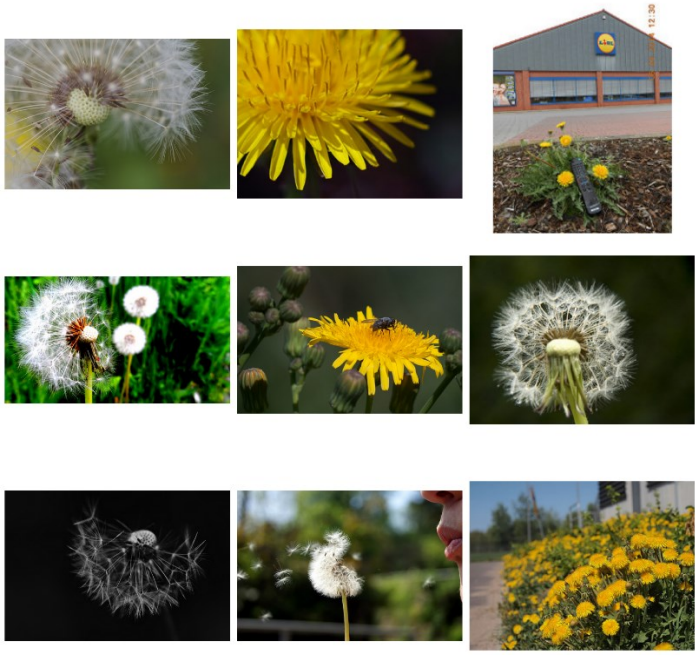
```
img_height, img_width = 150, 150
batch_size = 16
train_datagen = ImageDataGenerator(
    rescale=1.0/255,
    validation_split=0.2
)
train_generator = train_datagen.flow_from_directory(
    data_train,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='training'
```

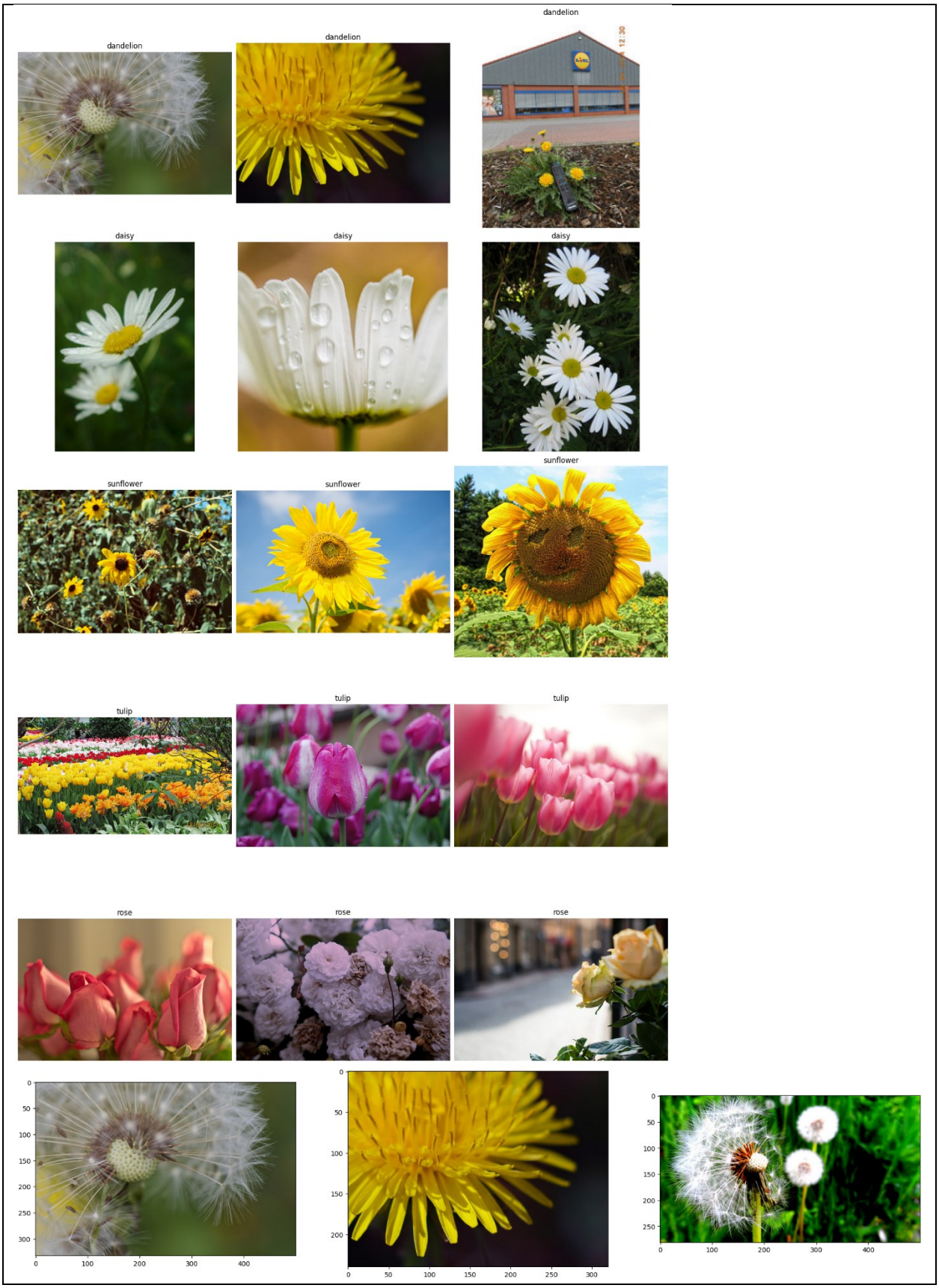
```
)
validation_generator = train_datagen.flow_from_directory(
    data_train,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation'
)
train_datagen = ImageDataGenerator(
    rescale=1.0/255,
    rotation_range=40,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    validation_split=0.2)
input_shape=(img_height, img_width, 3)
num_classes = train_generator.num_classes
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(img_height, img_width, 3),
activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(num_classes, activation='softmax'))
# Hyperparameters for tuning
optimizer = 'rmsprop'
learning_rate = 0.001
regularization = l2(0.01)
optimizer = 'adam'
learning_rate = 0.001
regularization = l1(0.01)
dropout_rate = 0.01
```

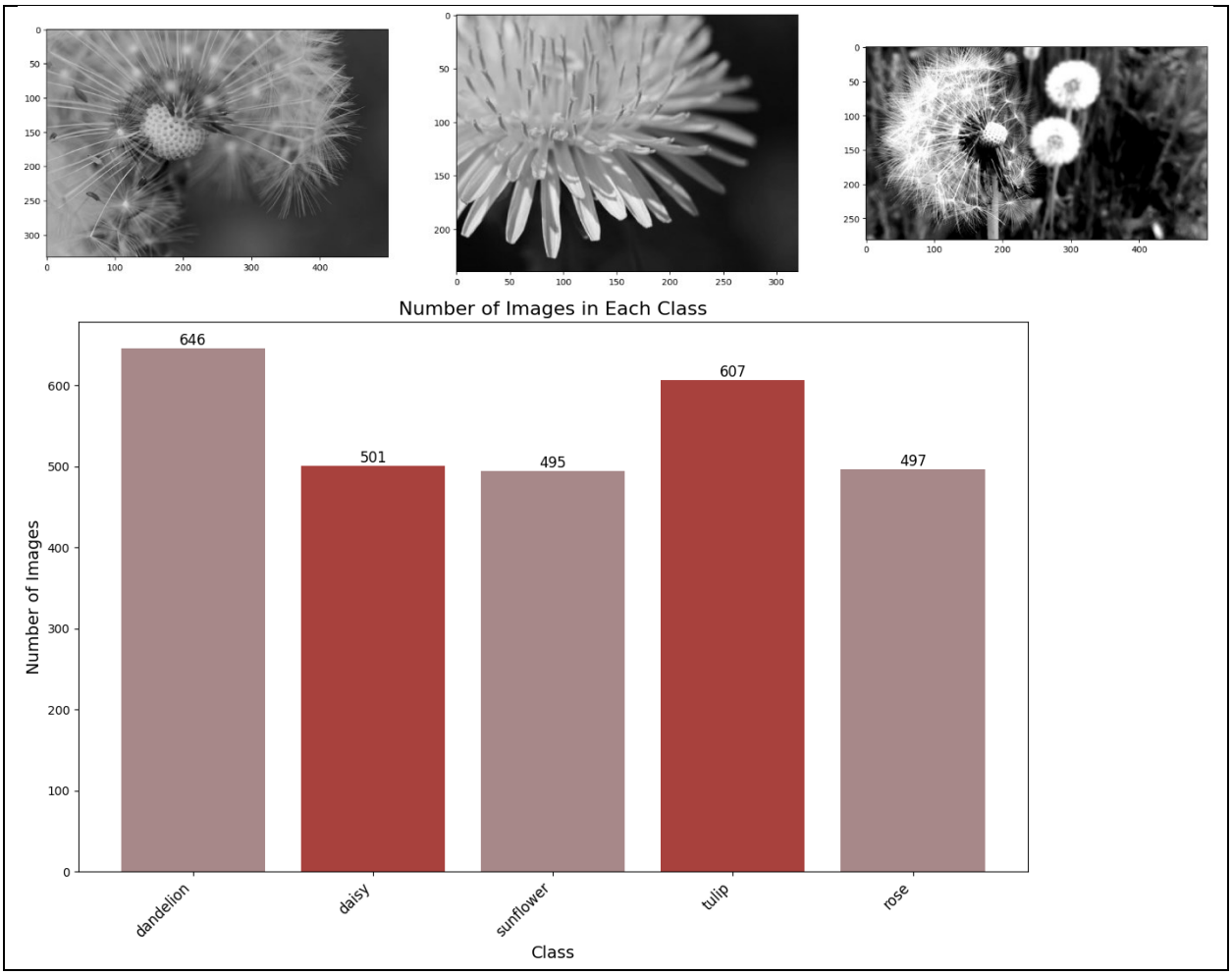
```
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.summary()
early_stopping = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
In [150]:
linkcode
history = model.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=10,
    callbacks=[early_stopping],
    verbose = 1
)
val_loss, val_accuracy = model.evaluate(validation_generator)
print(f'Validation Loss: {val_loss:.4f}, Validation Accuracy: {val_accuracy:.4f}')
validation_loss, validation_accuracy = model.evaluate(validation_generator)
print(f"Validation Accuracy: {validation_accuracy * 100:.2f}%")
plt.figure(figsize=(12, 5))
# Plot training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='upper left')
# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()
```

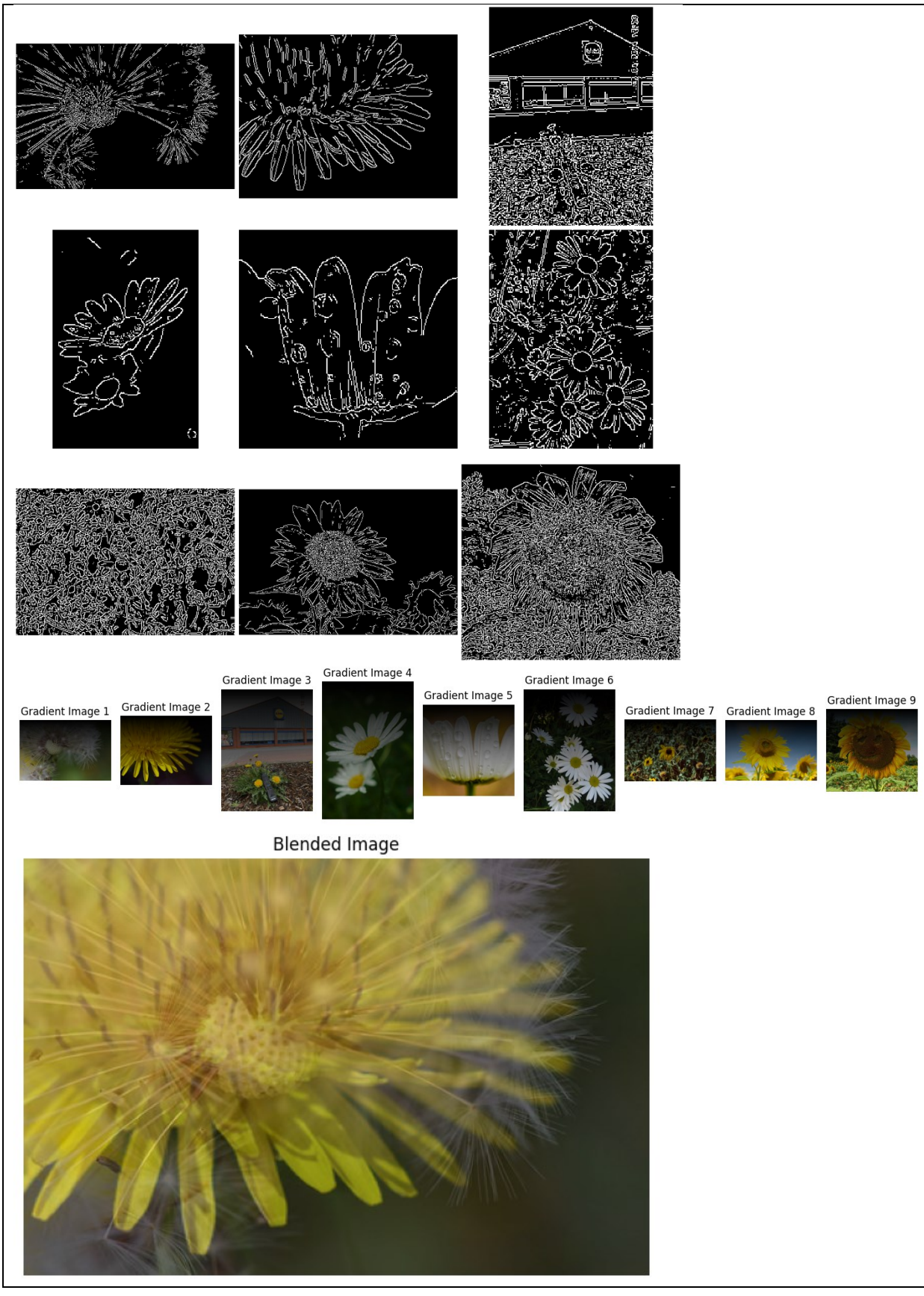
The results (Screenshot)

```
Number of dandelion images: 646
Number of daisy images: 501
Number of sunflower images: 495
Number of tulip images: 607
Number of rose images: 497
```

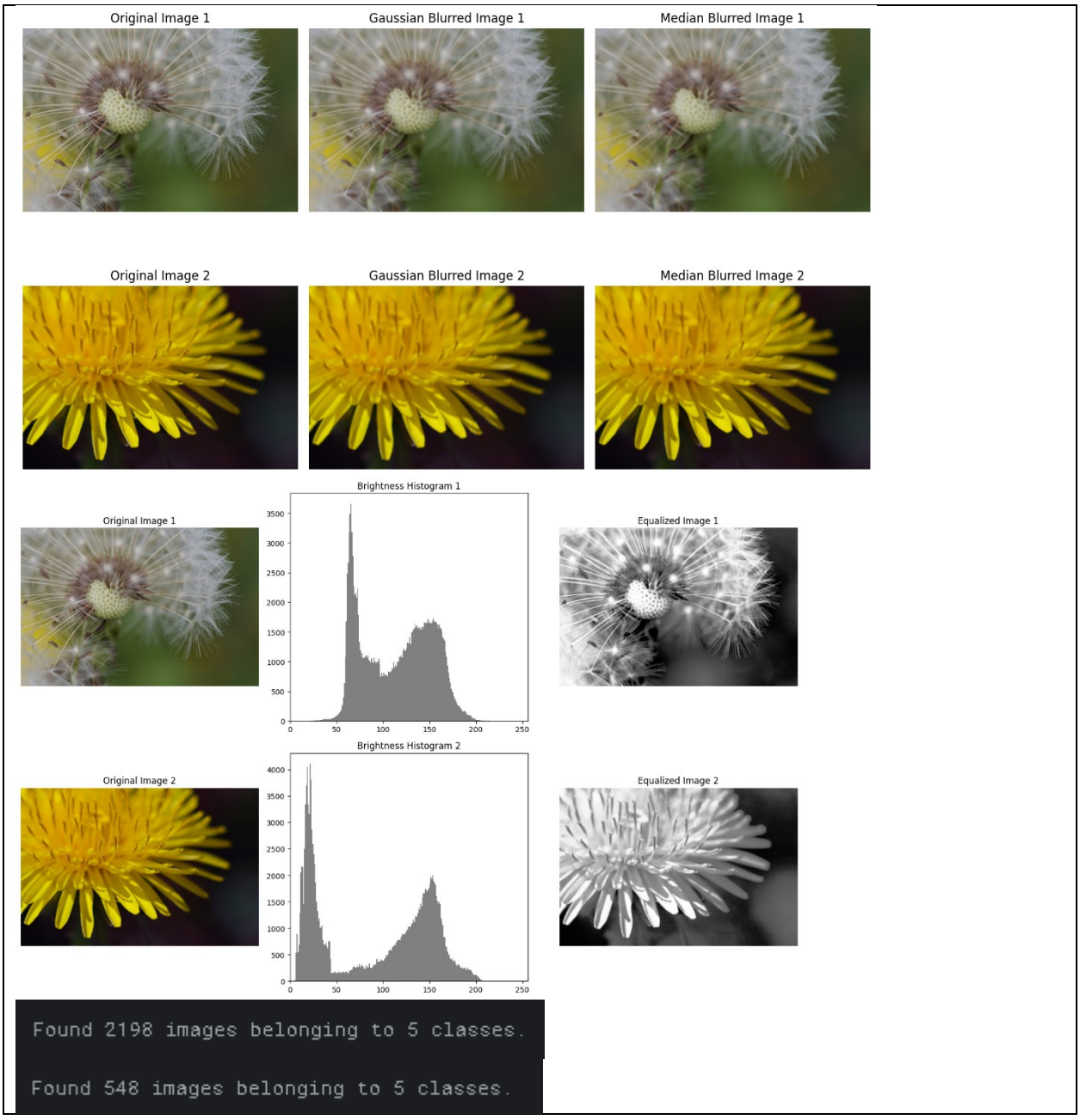



dandelion
dandelion
dandelion
daisy
daisy
daisy
sunflower
sunflower
sunflower
tulip
tulip
tulip
rose
rose
rose


Number of Images in Each Class
646
501
495
607
497
Number of Images
Class
dandelion
daisy
sunflower
tulip
rose


Gradient Image 1
Gradient Image 2
Gradient Image 3
Gradient Image 4
Gradient Image 5
Gradient Image 6
Gradient Image 7
Gradient Image 8
Gradient Image 9
Blended Image


Original Image 1
Adjusted Image 1
Original Image 2
Adjusted Image 2
Original Image 1
Zoomed Image 1
Shrunk Image 1
Original Image 2
Zoomed Image 2
Shrunk Image 2


Original Image 1
Gaussian Blurred Image 1
Median Blurred Image 1
Original Image 2
Gaussian Blurred Image 2
Median Blurred Image 2
Brightness Histogram 1
Original Image 1
Equalized Image 1
Brightness Histogram 2
Original Image 2
Equalized Image 2
Found 2198 images belonging to 5 classes.
Found 548 images belonging to 5 classes.

Model: "sequential_7"

Layer (type)	Output Shape	Param #
conv2d_17 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_17 (MaxPooling2D)	(None, 74, 74, 32)	0
dropout_6 (Dropout)	(None, 74, 74, 32)	0
conv2d_18 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_18 (MaxPooling2D)	(None, 36, 36, 64)	0
dropout_7 (Dropout)	(None, 36, 36, 64)	0
flatten_6 (Flatten)	(None, 82944)	0
dense_10 (Dense)	(None, 128)	10,616,960
dropout_8 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 5)	645

Total params: 10,636,997 (40.58 MB)

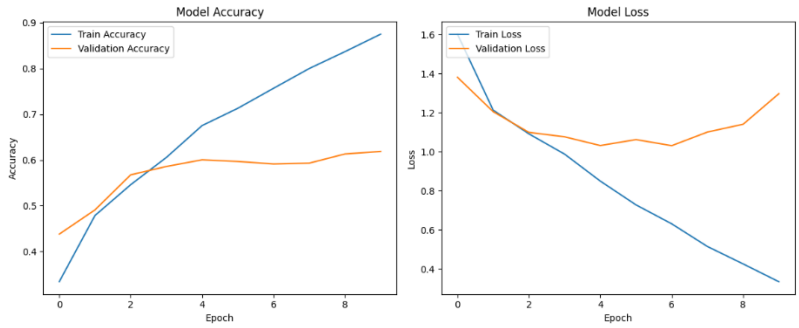
Trainable params: 10,636,997 (40.58 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 60s 416ms/step - accuracy: 0.2753 - loss: 2.1454 - val_accuracy:
0.4380 - val_loss: 1.3814
Epoch 2/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 57s 411ms/step - accuracy: 0.4610 - loss: 1.2335 - val_accuracy:
0.4909 - val_loss: 1.2051
Epoch 3/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 57s 411ms/step - accuracy: 0.5453 - loss: 1.0985 - val_accuracy:
0.5675 - val_loss: 1.0991
Epoch 4/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 57s 411ms/step - accuracy: 0.6175 - loss: 0.9940 - val_accuracy:
0.5858 - val_loss: 1.0764
Epoch 5/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 57s 412ms/step - accuracy: 0.6672 - loss: 0.8969 - val_accuracy:
0.6004 - val_loss: 1.0316
Epoch 6/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 57s 408ms/step - accuracy: 0.7169 - loss: 0.7320 - val_accuracy:
0.5967 - val_loss: 1.0620
Epoch 7/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 58s 415ms/step - accuracy: 0.7624 - loss: 0.6262 - val_accuracy:
0.5912 - val_loss: 1.0310
Epoch 8/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 56s 406ms/step - accuracy: 0.8051 - loss: 0.5088 - val_accuracy:
0.5931 - val_loss: 1.1007
Epoch 9/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 56s 404ms/step - accuracy: 0.8311 - loss: 0.4214 - val_accuracy:
0.6131 - val_loss: 1.1407
Epoch 10/10
130/130 ━━━━━━━━━━━━━━━━━━━━ 58s 417ms/step - accuracy: 0.8710 - loss: 0.3311 - val_accuracy:
0.6186 - val_loss: 1.2973

35/35 ━━━━━━━━━━━━━━━━━━━━ 4s 98ms/step - accuracy: 0.6160 - loss: 0.9784
Validation Loss: 1.0310, Validation Accuracy: 0.5912
```

```
35/35 ━━━━━━━━━━━━━━━━━━━━ 3s 95ms/step - accuracy: 0.6237 - loss: 0.9782
Validation Accuracy: 59.12%
```



Conclusion

In conclusion, we implemented the transfer learning training models by using Kaggle. And we the train model VGG16,ImageNet and the tensorflow library. Also, we use the cat and dog dataset and flower dataset in the Kaggle ide by using VG16 and ImageNet by training model of the AI.